Hafta 3 — Derste Yapılacak Uygulamalar

> **Ortam:** Sınıfta öğrenci bilgisayarı **yok**. Tüm uygulamalar **hoca ekranında canlı** yapılır; öğrenci tahmin eder, hoca çalıştırıp doğrular. Kodlama öğrenciye **ev görevi** olarak devredilir. **Toplam süre:** 48 dakika — blok sürelerinin **içindedir**, ek süre değildir (bkz. `H03-anlatim.md` bölüm 2) **Gerekli:** Projeksiyon, terminal, tarayıcı, Hafta 2'nin `KampusKitap` projesi. İnternetsiz yedek planlar aşağıda.

🔧 Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- [ ] `KampusKitap` projesi `dotnet run` ile açılıp kapatıldı
- [ ] **Hafta 2 hâlinin bir kopyası** alındı (canlı değişiklikler geri alınacak)
- [ ] Editör yazı tipi **≥ 18 punto**; `.cshtml` renklendirmesi çalışıyor
- [ ] Tarayıcıda **Ctrl+U** (kaynağı görüntüle) denendi — Uygulama A'nın kanıtı bu
- [ ] `Models/Kitap.cs` ve `Data/KitapDeposu.cs` projede **hazır** (aşağıda; derste yazılmaz, gösterilir)
- [ ] `H03-etkilesimli.html` açıldı, `N` denendi ve **kapatıldı**

> 💡 **Zaman kazancı:** `Kitap` modeli ve bellekteki `KitapDeposu` derste **yazılmaz**. Projede hazır durur, 30 saniyede gösterilir. Dersin konusu model değil, **görünüm**.

```
// Models/Kitap.cs — derste yazılmaz, hazır
namespace KampusKitap.Models;

public class Kitap
{
    public int Id { get; set; }
    public string Baslik { get; set; } = "";
    public string Yazar { get; set; } = "";
    public decimal Fiyat { get; set; }
}
```

Uygulama A — `Content()`'ten `View()`'a

Süre: 12 dakika · **Blok:** A **Amaç:** Razor'ın **sunucuda** çalıştığını kanıtlamak ve `return View()`'ın hangi dosyayı bulduğunu canlı göstermek.

Adım 1 — Hafta 2'nin hâlini hatırlat (1 dk)

`Controllers/KitapController.cs` içindeki mevcut satır:

```
public IActionResult Index()
{
    return Content("Kitap listesi");
}
```

Çalıştırın, `/Kitap` adresini açın: ekranda süslemesiz bir satır metin.

> **Sınıfa sor:** *"Bu bir web sayfası mı?"* Cevap: HTTP olarak evet, HTML olarak hayır. `Content()` düz metin gönderir.

Adım 2 — `View()`'a çevir ve **hata al** (3 dk)

```
public IActionResult Index()
{
    return View();
}
```

> **Sınıfa sor (çalıştırmadan):** *"Dosyayı daha yazmadım. Ne olacak?"* Beklenen: "boş sayfa", "hata". **Hata gelecek — ve o hata dersin hediyesi.**

Çalıştırın. Ekranda:

```
InvalidOperationException: The view 'Index' was not found.
The following locations were searched:
/Views/Kitap/Index.cshtml
/Views/Shared/Index.cshtml
```

⛔ **Bu ekranı atlamayın.** Yol listesini tek tek okutun. Ders notu bölüm 2.5 ve 🐛 Hata Köşesi tamamen bu listedir. Öğrenciye söyleyeceğiniz cümle:

> *"Çatı size nerede aradığını söylüyor. Hatanın çözümü hatanın içinde yazıyor."*

Adım 3 — Görünümü yaz (4 dk)

`Views/Kitap/Index.cshtml` — **listedeki birinci yol**:

```
@model List<KampusKitap.Models.Kitap>

<h1>Kitaplar</h1>

@if (Model.Count == 0)
{
    <p>Hiç kitap yok.</p>
}
else
{
    <ul>
        @foreach (var k in Model)
        {
            <li><strong>@k.Baslik</strong> — @k.Yazar — @k.Fiyat TL</li>
        }
    </ul>
}
```

Controller'da modeli gönderin:

```
public IActionResult Index()
{
    return View(KitapDeposu.Hepsi);
}
```

Çalıştırın. Liste geldi.

⚠️ **Sık hata:** `View()` demek ama modeli göndermemek → görünümde `Model` `null` olur, `Model.Count` `NullReferenceException` fırlatır. Bunu **bilerek bir kez yapın**, sonra düzeltin.

Adım 4 — ⭐ Kanıt: Ctrl+U (4 dk)

> **Sınıfa sor (açmadan önce):** "*Şimdi sayfanın kaynağını açacağım.* `@foreach` *satırını görecek miyiz?*"
>
> Elleri saydırın. Genellikle sınıfın yarısı "evet" der.

Ctrl+U ile kaynağı açın. Ekranda:

```
<h1>Kitaplar</h1>
<ul>
    <li><strong>Suç ve Ceza</strong> — Dostoyevski — 120 TL</li>
    <li><strong>Tutunamayanlar</strong> — Oğuz Atay — 180 TL</li>
</ul>
```

`@model` yok. `@foreach` yok. `@if` yok.

> **Söylenecek cümle:** *"Razor tarayıcıya gitmedi. Sunucuda çalıştı, işi bitti, ortadan kayboldu. Tarayıcıya yalnızca sonucu gönderdi. Bu yüzden Razor'ı tarayıcının konsolunda hata ayıklayamazsınız — orada yok."*

Yedek plan: Projeksiyonda kaynak kodu küçük kalıyorsa Ctrl+U yerine sayfada sağ tık → *İncele* kullanmayın (DOM gösterir, kaynak değil). Kaynağı görüntüleyip tarayıcı yakınlaştırmasını (Ctrl +) iki kademe artırın.

Uygulama B — Layout'u Özelleştirmek ve `@section`

Süre: 14 dakika · **Blok:** B **Amaç:** `@RenderBody()`'nin ne yaptığını **silerek** göstermek; `_ViewStart`'ın işini kanıtlamak; `@section`'ı kurmak.

Adım 1 — `@RenderBody()`'yi bul ve **sil** (4 dk)

`Views/Shared/_Layout.cshtml` dosyasını açın, `@RenderBody()` satırını bulun.

> **Sınıfa sor:** *"Geçen hafta bu satırı sormuştum. Tahminlerinizi tahtaya yazmıştık. Şimdi bu satırı siliyorum. Ne olacak?"*
>
> Beklenen cevaplar: "hata", "boş sayfa", "sadece menü kalır".

Satırı **yorum satırı yapın** (`@* … *@` ile) ve çalıştırın.

Sonuç: menü ve footer **var**, kitap listesi **yok**. Hata **yok**.

> **Söylenecek cümle:** *"Kitaplar kayboldu ama sayfa çalışmaya devam etti. Demek ki `@RenderBody()` sayfanın kendisi değil, içeriğin **yerleşeceği yuva**."*

Geri alın.

Adım 2 — Menüyü değiştir, üç sayfada birden değiştiğini göster (3 dk)

Layout'taki menüye bir bağlantı ekleyin:

```
<a href="/Kitap">Kitaplar</a>
<a href="/Kategori">Kategoriler</a>
<a href="/Home/Privacy">Gizlilik</a>   @* ← yeni *@
```

`/Kitap`, `/Kategori`, `/` adreslerini sırayla açın: **üçünde de** yeni bağlantı var.

> 🧩 **İlke burada bağlanır:** *"Tek dosyayı değiştirdim, üç sayfa değişti. Eğer bu menü üç dosyaya kopyalanmış olsaydı, birini unuttuğumda kimse fark etmezdi."*

Adım 3 — `_ViewStart.cshtml`'i kapat (3 dk)

`Views/_ViewStart.cshtml` içeriğini yorum satırı yapın:

```
@*
@{
    Layout = "_Layout";
}
*@
```

> **Sınıfa sor:** "*Bu dosyayı kapattım. Sayfa ne olacak?*"

Çalıştırın: menü, CSS, footer **gitti**; yalnızca `<h1>Kitaplar</h1>` ve liste kaldı. Tarayıcı varsayılan yazı tipiyle çıplak HTML.

> **Söylenecek cümle:** "*Layout bağlantısını hiçbir görünümde yazmamıştık. Bu dosya her görünümden önce çalışıp bizim yerimize yazıyormuş.*"

Geri alın.

Adım 4 — `@section` ekle (4 dk)

Layout'ta `</body>` üstünde zaten var olan satırı gösterin:

```
@await RenderSectionAsync("Scripts", required: false)
```

`Views/Kitap/Index.cshtml` sonuna ekleyin:

```
@section Scripts {
    <script>
        console.log("Yalnızca kitap listesinde çalışan betik");
    </script>
}
```

Tarayıcı konsolunu açın: `/Kitap`'ta mesaj **var**, `/Kategori`'de **yok**.

> **Bonus (süre varsa):** `required: false`'u `true` yapın → `/Kategori` **hata verir**, çünkü o görünüm bölümü doldurmuyor. Geri alın.

⚠️ **Sık hata:** Bölüm adı büyük/küçük harfe duyarlıdır. `@section scripts` yazarsanız layout'taki `"Scripts"` ile eşleşmez ve "*sections have been defined but have not been rendered*" hatası gelir.

Uygulama C — Kartı Partial'a Çıkarmak

Süre: 10 dakika · **Blok:** C **Amaç:** Tekrar eden **parçayı** tek sahibe bağlamak; partial'ın veriyi **dışarıdan** aldığını göstermek.

Adım 1 — Tekrarı göster (2 dk)

`Views/Kitap/Index.cshtml` ve `Views/Kitap/Ara.cshtml` dosyalarında **aynı kart** duruyor:

```
<article class="kart">
    <h3>@k.Baslik</h3>
    <p>@k.Yazar</p>
    <p class="fiyat">@k.Fiyat TL</p>
</article>
```

> **Sınıfa sor:** "*Karta 'stokta yok' etiketi eklemem gerekse kaç dosya açarım?*"

Adım 2 — Partial'a çıkar (4 dk)

`Views/Shared/_KitapKart.cshtml`:

```
@model KampusKitap.Models.Kitap

<article class="kart">
    <h3>@Model.Baslik</h3>
    <p>@Model.Yazar</p>
    <p class="fiyat">@Model.Fiyat TL</p>
</article>
```

İki görünümde de yerine koyun:

```
@foreach (var k in Model)
{
    <partial name="_KitapKart" model="k" />
}
```

Çalıştırın: görüntü **aynı**. Kod üç yerden bire indi.

Adım 3 — `model="k"` sil, hatayı göster (2 dk)

> **Sınıfa sor:** "`model="k"` *yazmazsam ne olur? Partial neyi basar?*"

Silin ve çalıştırın:

```
InvalidOperationException: The model item passed into the ViewDataDictionary is of type
'System.Collections.Generic.List`1[KampusKitap.Models.Kitap]', but this ViewDataDictionary
instance requires a model item of type 'KampusKitap.Models.Kitap'.
```

> **Söylenecek cümle:** *"Model vermezsem partial **üst görünümün** modelini alır. Yani partial veriyi kendi getirmiyor — ona ne verirsem onu basıyor. Bu cümleyi aklınızda tutun; birazdan tam tersini yapan bir şey göreceğiz."*

Geri alın.

Adım 4 — Değişikliği tek yerden yap (2 dk)

`_KitapKart.cshtml`'e tek satır ekleyin:

```
<span class="rozet">kampüs içi</span>
```

İki sayfada birden değişti. 🧩 İlke ikinci kez bağlandı.

Uygulama D — `IZiyaretSayaci`'yı View Component'e Dönüştürmek

Süre: 12 dakika · **Blok:** C **Amaç:** Partial'ın yapamadığını göstermek: **kendi verisini getiren** parça. Hafta 2'de kurulan DI'ın aynen çalıştığını kanıtlamak.

Adım 1 — Problemi kur (2 dk)

> **Sınıfa sor:** *"Hafta 2'de `IZiyaretSayaci` yapmıştık. Şimdi bu sayacı **her sayfanın altında** göstermek istiyorum. Partial ile yapsam ne gerekir?"*

Cevabı tahtaya yazın:

```
HomeController     → sayacı al, ViewBag'e koy
KitapController    → sayacı al, ViewBag'e koy
KategoriController → sayacı al, ViewBag'e koy
```

> *"Üç controller aynı kodu yazdı. Yarın dördüncüyü eklediğimde unutacağım ve o sayfada rozet görünmeyecek. Hata da almayacağım."*

Adım 2 — View Component sınıfını yaz (4 dk)

`ViewComponents/ZiyaretRozetiViewComponent.cs`:

```
using Microsoft.AspNetCore.Mvc;
using KampusKitap.Services;

namespace KampusKitap.ViewComponents;

public class ZiyaretRozetiViewComponent : ViewComponent
{
    private readonly IZiyaretSayaci _sayac;

    // Hafta 2'nin kurucu enjeksiyonu — hiçbir şey değişmedi.
    public ZiyaretRozetiViewComponent(IZiyaretSayaci sayac)
    {
        _sayac = sayac;
    }

    public IViewComponentResult Invoke()
    {
        var deger = _sayac.Arttir();   // ← VERİYİ KENDİ GETİRİYOR
        return View(deger);
    }
}
```

> **Vurgulayın:** `Controller` değil, `ViewComponent` taban sınıfı. Route'lanmaz; adresi yoktur. Ama DI aynı DI.

Adım 3 — Görünümünü yaz — **yolu kasten yanlış yap** (3 dk)

Önce yanlış yere koyun: `Views/Shared/ZiyaretRozeti.cshtml`

Layout'a ekleyin:

```
<footer>
    © 2026 KampusKitap — <vc:ziyaret-rozeti />
</footer>
```

Çalıştırın → görünüm bulunamadı hatası, aranan yollar listesiyle.

Şimdi **doğru yola** taşıyın:

```
Views/Shared/Components/ZiyaretRozeti/Default.cshtml
```

```
@model int
<span class="rozet">Bu sayfa @Model kez gösterildi</span>
```

> **Söylenecek cümle:** *"Klasör adı sınıf adının `ViewComponent` soneki atılmış hâli. Dosya adı `Default.cshtml`. Bu bir öneri değil, sözleşme — Hafta 2'deki `Controller` soneki sözleşmesiyle **aynı mantık**."*

Adım 4 — Kanıtla: hiçbir controller'a dokunmadık (3 dk)

`/`, `/Kitap`, `/Kategori` adreslerini sırayla açın. Üçünde de rozet var, sayı artıyor.

> **Sınıfa sor:** *"Kaç controller düzenledim?"* Cevap: **sıfır**.
>
> **Kapanış cümlesi:** *"Partial veriyi **alır**. View Component veriyi **kendi getirir**. Sınavda bu ayrımı soracağım."*

Yedek plan (süre bitti): Adım 3'ün yanlış-yol denemesini atlayın, doğrudan doğru yola yazın. Adım 4 asla atlanmaz — uygulamanın bütün anlamı orada.

🏠 Ders Sonrası Öğrenci Görevi

1. Kendi projenizde **Layout**'u kurun; üst menüyü layout'a taşıyın. En az **üç** sayfada aynı menünün göründüğünü ekran görüntüsüyle gösterin.
2. Menüyü ayrıca `_Menu.cshtml` adlı bir **partial**'a çıkarın ve layout'tan çağırın.
3. Tekrar eden bir **bilgi kartını** partial'a çıkarın (`model="…"` ile veri geçirin).
4. **Bir View Component yazın**: kendi verisini getiren bir parça (örn. "toplam kayıt sayısı" rozeti). Hiçbir controller'a dokunmadan **iki farklı sayfada** göründüğünü kanıtlayın.
5. `@RenderBody()` satırını geçici olarak silin, ekran görüntüsünü alın, geri koyun.
6. Görünümünüzde bir alana `<b>kalın</b>` metnini veri olarak verin; `@Model.X` ile `@Html.Raw(Model.X)` çıktılarının **farkını** ekran görüntüsüyle gösterin.

Teslim: proje klasörü + 6 maddenin ekran görüntüleri + şu soruya **iki cümlelik** yazılı cevap: *"4. maddede neden partial değil View Component kullandınız?"*

✅ Hocaya Kontrol Listesi

	Uygulama	Kritik an	Atlanabilir mi
A	`Content()` → `View()`	**Adım 2 hata ekranı** + **Adım 4 Ctrl+U**	⛔ hayır
B	Layout ve section	**Adım 1** `@RenderBody()` silme	Adım 4 bonusu atlanabilir
C	Partial	Adım 3 model hatası	Adım 3 atlanabilir
D	View Component	**Adım 4** — sıfır controller	⛔ hayır

Her uygulamada sıra aynıdır: önce **tahmin ettir**, sonra **çalıştır**. Tersi yapılırsa etkinin tamamı kaybolur.

⚠️ Yedek Planların Özeti

Sorun	Yedek plan
Proje derlenmiyor	`Hafta-03/KampusKitap` **bitmiş hâli** hazır — oradan devam edin, canlı yazmayı anlatıya çevirin
`dotnet run` port hatası veriyor	`dotnet run --urls http://localhost:5080`
HTTPS sertifika uyarısı	`dotnet dev-certs https --trust` (ders öncesi yapılmış olmalı)
Projeksiyonda kaynak kodu okunmuyor	Ctrl + ile tarayıcıyı iki kademe yakınlaştırın
Süre taştı	Uygulama C'yi tümden atlayın; partial'ı **bitmiş hâlden gösterin**
Tarayıcı `.cshtml` değişikliğini görmüyor	`dotnet run`'ı yeniden başlatın (Hot Reload her değişikliği yakalamaz)